Architecture Description

Warehouse Robots – Team 3

Software Engineering for Cyber-Physical Systems

2025–2026

Table of Contents

1. Introduction 3
2. Requirements 3
  2.1 World–Machine Model 3
  2.2 Functional Requirements 3
  2.3 Stimulus → Response Specification 4
3. Evolution from Initial Design 4
4. System Overview 5
5. Robot Architecture 6
  5.1 State Machine 6
  5.2 Hardware Abstraction & Dependency Injection 7
  5.3 Navigation 8
  5.4 Odometry (Sensor Fusion) 8
  5.5 Collision Avoidance 8
6. Server Architecture 9
  6.1 Server Update Loop (100 ms) 9
  6.2 Component Summary 9
7. Communication Architecture 10
  7.1 Task Sequence 10
  7.2 MQTT Topics (QoS = 1, JSON) 10
  7.3 REST API (FastAPI) 11
  7.4 WebSocket (/ws) 11
8. Formal Specifications (TLA+) 12
  8.1 Aisle Mutual Exclusion 12
  8.2 Robot State Machine 12
  8.3 Task Lifecycle 13
9. Key Design Choices 14
10. Conclusion 14

1. Introduction

This document describes the software architecture of the Warehouse Robots system (Team 3). The system coordinates a fleet of autonomous mobile robots that pick items from warehouse shelves and return them to a central base. Three architectural goals guided the design:

- Separation of concerns: robot control, server orchestration, and the web interface are independent layers
- Platform independence: robot logic is decoupled from Webots via abstract interfaces
- Resilience: lease-based locking, offline detection, and priority-based collision avoidance handle partial failures

2. Requirements

2.1 World–Machine Model

The environment and system phenomena are classified according to the World–Machine model:

Category	Phenomena
World (physical env)	Warehouse aisles (one robot at a time), shelves, base location, mobile robots, WiFi network, human operators
Hidden env. phenomena (eh)	Mechanical wear, human presence in aisles, WiFi interference, hardware failures, floor obstacles
Visible env. phenomena (ev)	LiDAR distance readings, wheel encoder ticks, compass yaw, robot network connectivity
Visible system phenomena (sv)	Motor velocity commands, aisle lock grant/release messages, task assignment commands, heartbeat messages, task status updates

2.2 Functional Requirements

#	Requirement	Property
1	Liveness	Every queued task is eventually picked and delivered
2	Collision freedom	Robots never collide with each other or obstacles
3	Aisle exclusivity	At most one robot is present in any aisle at any time
4	Return to base	Every robot returns to the base after each pick
5	Operator interface	Operator can manage the queue and monitor robots via web UI
6	Autonomous recovery	System recovers from connectivity loss without operator intervention
7	Portability	Deployable on real hardware with minimal code changes

2.3 Stimulus → Response Specification

Stimulus (ev)	Response (sv)
Distance reading < stop threshold	Immediate stop command issued to robot
Robot connectivity → offline	Motion stops; aisle lease expires server-side after 60 s
Robot requests aisle entry	Aisle marked locked; other robots denied access
Operator submits task via web UI	Navigation command issued to nearest available robot
Aisle lease expires (60 s without release)	Lock reclaimed; owning robot notified with granted=false

3. Evolution from Initial Design

Key differences between the initial design (first presentation) and the current implementation:

Initial design	Current implementation	Reason
Shortest-path navigation algorithm	Proportional controller to fixed coordinates	Static layout makes path planning unnecessary
Local mapping as a system phenomenon	Static WarehouseMap with hardcoded coordinates	Fixed geometry; no runtime map needed
Microservices architecture	Single-process FastAPI with modular internal structure	Reduced operational complexity; modules still separate concerns
Strict hexagonal (ports & adapters)	Layered architecture with dependency inversion (DIP)	DIP fully applied; strict port naming not needed at this scale
Explicit safe state on connectivity loss	WAIT_CONNECTION only at startup; lease expires server-side	Mid-task loss handled implicitly via lease timeout
Sensor processing pipeline	Single front-sector LiDAR + compass	Complex fusion not needed in simulation

4. System Overview

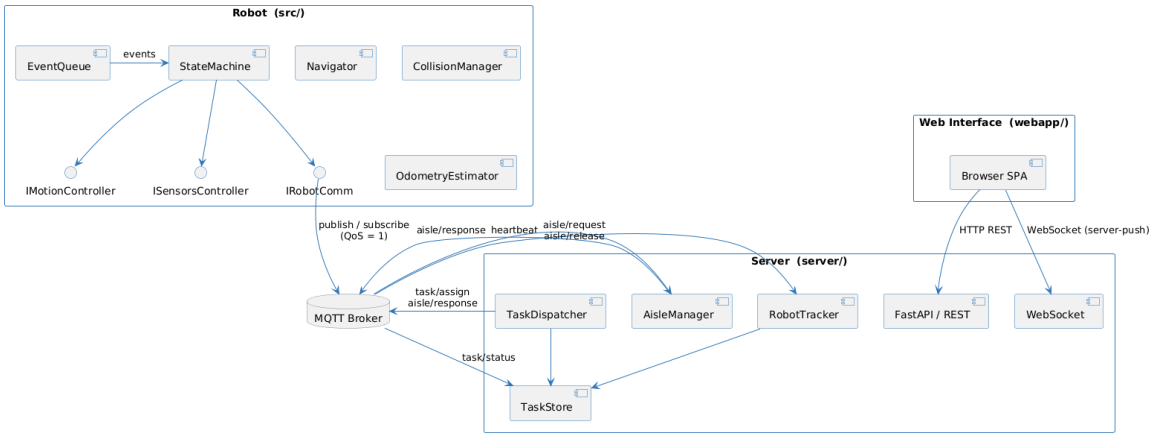

Figure 1 – Three-tier component architecture

Tier	Directory	Responsibility	Protocol
Robot	src/	State machine, navigation, sensor fusion, collision avoidance	MQTT
Server	server/	Task dispatch, aisle management, robot registry, API	MQTT + HTTP/WS
Web Interface	webapp/	Task submission, real-time fleet monitoring	REST + WebSocket

5. Robot Architecture

5.1 State Machine

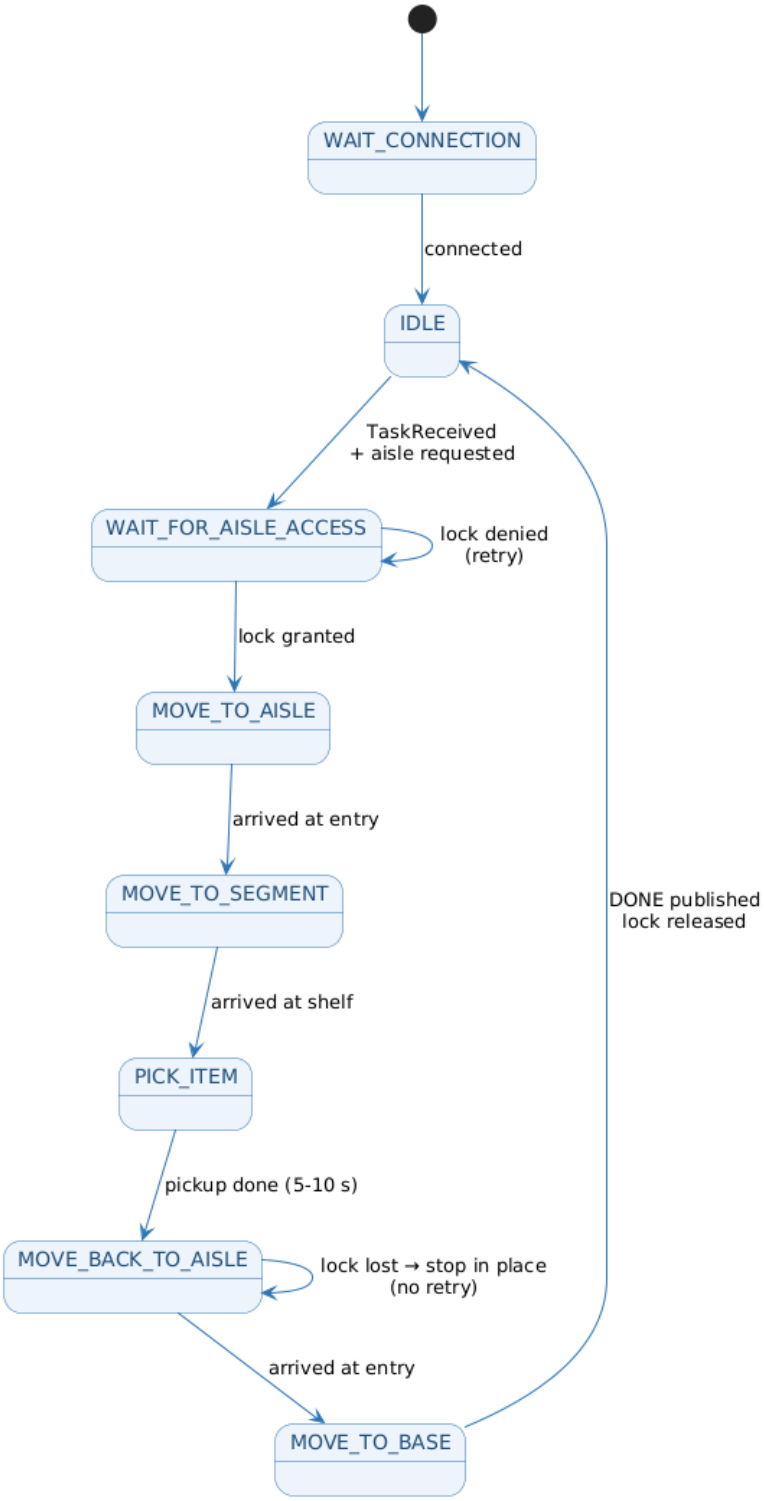

Figure 2 – Robot finite state machine

Each state implements IRobotState: on_enter(robot), on_exit(robot), update(robot, dt) → TransitionID (called every tick), and on_event(robot, event) → None (handles async MQTT events).

State	Behaviour
WAIT_CONNECTION	Connects to MQTT broker; transitions to IDLE on success
IDLE	Stops robot; on TaskReceivedEvent stores task, requests aisle lock, publishes IN_PROGRESS
WAIT_FOR_AISLE_ACCESS	Halts; retries until AisleResponseEvent(granted=true); sets aisle_lock_granted flag
MOVE_TO_AISLE	Navigates to aisle entry point; transitions on arrival
MOVE_TO_SEGMENT	Navigates to shelf location; transitions on arrival
PICK_ITEM	Stops; waits 5–10 s (simulated pickup); transitions to MOVE_BACK_TO_AISLE
MOVE_BACK_TO_AISLE	Returns to entry; if aisle_lock_granted is false → stop in place (does not re-request)
MOVE_TO_BASE	Releases lock, navigates to base, publishes DONE, transitions to IDLE

5.2 Hardware Abstraction & Dependency Injection

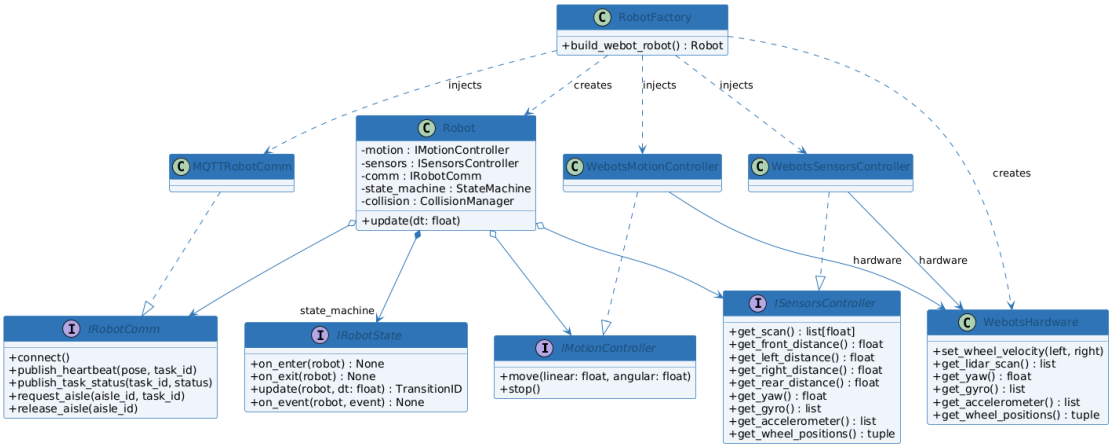

Figure 3 – Interface hierarchy and dependency injection via RobotFactory

The Robot class depends only on interfaces. RobotFactory creates a WebotsHardware instance (the low-level Webots device layer) and passes it to both WebotsMotionController and WebotsSensorsController. These controllers translate abstract interface calls into Webots device API calls. Collision avoidance is applied in each state's update() via robot.collision.apply(), not inside the motion controller.

ISensorsController exposes: get_scan(), get_front_distance(), get_left_distance(), get_right_distance(), get_rear_distance(), get_yaw(), get_gyro(), get_accelerometer(), get_wheel_positions(). Replacing WebotsHardware, WebotsMotionController, and WebotsSensorsController is sufficient for real-hardware deployment.

5.3 Navigation

Proportional controller (no PID); arrival threshold 0.02 m:

```
angle_error  = normalize(atan2(dy, dx) − pose.theta)
angular_vel  = clamp(k_angular × angle_error,  −max_ω, max_ω)
linear_vel   = min(k_linear × distance × cos(angle_error), max_v)
if |angle_error| > 0.8 rad:  linear_vel = 0   # rotate-in-place first
```

Parameters: k_linear = 1.0, k_angular = 2.5, max_linear = 0.20 m/s, max_angular = 1.5 rad/s.

5.4 Odometry (Sensor Fusion)

Wheel encoders provide position; compass overrides yaw to eliminate drift:

```
if |d_theta| < 1e-6:  x += d_center·cos θ;  y += d_center·sin θ   # straight-line
else:  x += r·(sin(θ+dθ) − sin θ);  y += r·(cos θ − cos(θ+dθ))  # circular arc
theta = compass_yaw  (if available)
```

5.5 Collision Avoidance

State	Trigger	Effect
SAFE	Path is clear	Full speed
EVADE	Obstacle detected and closing, or critically close	Reduced linear speed; left-turn bias added to navigator output
STOP	Obstacle critically close	Zero velocity; forced evasion window after recovery

The CollisionManager uses front-sector distance readings combined with a closing-rate calculation to distinguish between stationary obstacles and approaching robots. A mandatory evasion window after a hard stop prevents the robot from immediately resuming full speed before the obstacle has cleared.

6. Server Architecture

6.1 Server Update Loop (100 ms)

1. **Process MQTT events** → forward to RobotTracker, AisleManager, TaskStore; WebSocket bridge broadcasts immediately.
2. **RobotTracker.update()** → marks robots OFFLINE if no heartbeat for 4 s.
3. **AisleManager.update()** → expires leases > 60 s; notifies affected robot with granted=false.
4. **TaskDispatcher.try_dispatch()** → assigns next pending task to nearest idle robot.

6.2 Component Summary

Component	Responsibility
TaskDispatcher	FIFO task queue; nearest-idle-robot assignment (Euclidean distance to aisle entry)
AisleManager	Lease-based mutual exclusion (one robot per aisle); 60 s timeout for crash recovery
RobotTracker	Auto-registers robots on first heartbeat; OFFLINE after 4 s silence; OFFLINE state not cleared by heartbeat
TaskStore	Tracks full task lifecycle: PENDING → IN_PROGRESS → DONE / REJECTED
WarehouseMap	Static geometry: 5 aisles × 3 shelves; aisle entry X = 3.0 m; shelf X = 4.5 / 5.5 / 6.5 m

7. Communication Architecture

7.1 Task Sequence

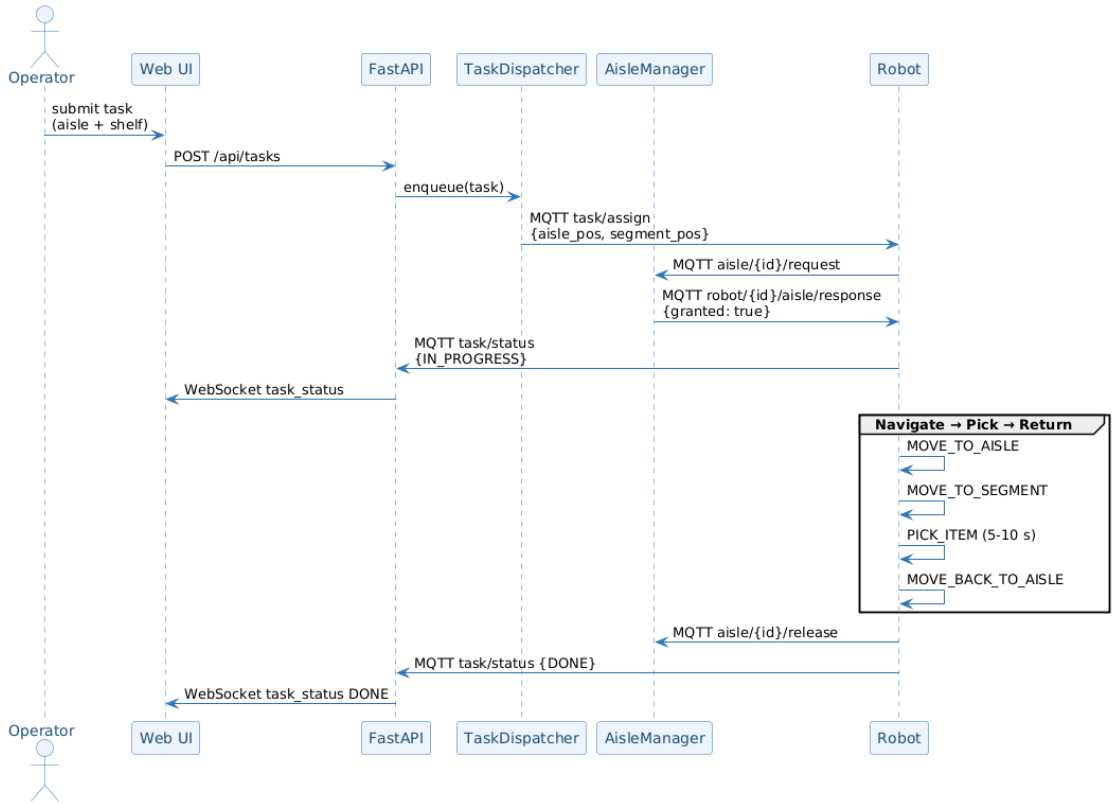

Figure 4 – End-to-end task sequence (Operator → Robot → Done)

7.2 MQTT Topics (QoS = 1, JSON)

Topic	Direction	Key payload fields
robot/{id}/heartbeat	Robot → Server	robot_id, pose {x,y,θ}, task_id
robot/{id}/task/assign	Server → Robot	task_id, aisle_pos, segment_pos
robot/{id}/task/status	Robot → Server	task_id, status, reason
robot/{id}/aisle/response	Server → Robot	aisle_id, granted
aisle/{id}/request	Robot → Server	robot_id, task_id
aisle/{id}/release	Robot → Server	robot_id

7.3 REST API (FastAPI)

Method + Path	Description	Response
POST /api/tasks	Submit new task	201 TaskSchema
GET /api/tasks	List tasks (filter by status)	200 TaskSchema[]
DELETE /api/tasks/{id}	Cancel PENDING task	204 / 409
GET /api/robots	All robots + state	200 RobotSchema[]
GET /api/aisles	All aisles + lock status	200 AisleSchema[]

7.4 WebSocket (/ws)

Server-push events to the browser: robot_heartbeat, task_status, aisle_request, aisle_release. Eliminates polling; WebSocketEventBridge converts domain events to JSON and broadcasts to all connected clients.

8. Formal Specifications (TLA+)

Three TLA+ modules capture the key safety and liveness properties.

8.1 Aisle Mutual Exclusion

```
---- MODULE AisleMutualExclusion ----
EXTENDS Integers, FiniteSets
CONSTANTS Robots, Aisles
VARIABLES aisle_owner, robot_state

NULL == CHOOSE n : n \notin Robots

TypeOK ==
    /\ aisle_owner \in [Aisles -> Robots \cup {NULL}]
    /\ robot_state \in [Robots -> {"IDLE","WAIT_AISLE","IN_AISLE","RETURNING"}]

(* Safety: at most one robot holds any aisle lock *)
MutualExclusion ==
    \A a \in Aisles : Cardinality({r \in Robots : aisle_owner[a] = r}) <= 1

RequestAisle(r, a) ==
    /\ robot_state[r] = "WAIT_AISLE" /\ aisle_owner[a] = NULL
    /\ aisle_owner' = [aisle_owner EXCEPT ![a] = r]
    /\ robot_state' = [robot_state EXCEPT ![r] = "IN_AISLE"]

ReleaseAisle(r, a) ==
    /\ aisle_owner[a] = r
    /\ aisle_owner' = [aisle_owner EXCEPT ![a] = NULL]
    /\ robot_state' = [robot_state EXCEPT ![r] = "RETURNING"]

ExpireLease(a) ==  (* server reclaims lock after 60 s timeout *)
    /\ aisle_owner[a] # NULL
    /\ aisle_owner' = [aisle_owner EXCEPT ![a] = NULL]
    /\ UNCHANGED robot_state
====
```

8.2 Robot State Machine

```
---- MODULE RobotStateMachine ----
VARIABLES state, has_task, aisle_granted

States == {"WAIT_CONNECTION","IDLE","WAIT_FOR_AISLE_ACCESS",
           "MOVE_TO_AISLE","MOVE_TO_SEGMENT","PICK_ITEM",
           "MOVE_BACK_TO_AISLE","MOVE_TO_BASE"}

Init == /\ state = "WAIT_CONNECTION" /\ has_task = FALSE /\ aisle_granted =
FALSE

(* Safety: robot never enters aisle without holding the lock *)
NeverEnterWithoutLock ==
    state \in {"MOVE_TO_AISLE","MOVE_TO_SEGMENT",
               "PICK_ITEM","MOVE_BACK_TO_AISLE"}
    => aisle_granted = TRUE

(* Liveness: every assigned task eventually completes *)
TaskEventuallyDone == has_task ~> (state = "IDLE" /\ ~has_task)
====
```

8.3 Task Lifecycle

```
---- MODULE TaskLifecycle ----
EXTENDS FiniteSets
CONSTANTS Tasks, Robots
NULL == CHOOSE n : n \notin Robots
VARIABLES task_status, task_robot

Init ==
    /\ task_status = [t \in Tasks |-> "PENDING"]
    /\ task_robot  = [t \in Tasks |-> NULL]

Dispatch(t, r) ==
    /\ task_status[t] = "PENDING" /\ task_robot[t] = NULL
    /\ task_status' = [task_status EXCEPT ![t] = "IN_PROGRESS"]
    /\ task_robot'  = [task_robot  EXCEPT ![t] = r]

(* Safety: a task is assigned to at most one robot *)
SingleAssignment ==
    \A t \in Tasks : task_status[t] = "IN_PROGRESS" => task_robot[t] # NULL

(* Safety: DONE and REJECTED are terminal states *)
NoRollback ==
    /\ [][task_status[t] = "DONE"     => task_status'[t] = "DONE"]_task_status
    /\ [][task_status[t] = "REJECTED" => task_status'[t] =
"REJECTED"]_task_status
====
```

9. Key Design Choices

Design Choice	Alternative	Rationale
State machine for robot logic	Monolithic loop	Single responsibility per state; extensible without modifying existing states
Abstract interfaces + factory (DIP)	Direct Webots dependency	Robot core has zero Webots imports; swap two classes for real hardware
MQTT pub-sub (QoS = 1)	Direct TCP / HTTP polling	Scales to N robots; handles WiFi dead spots; wildcard subscriptions
Lease-based aisle locking (60 s)	Full path planning	Simple crash recovery; no extra protocol needed
Priority-based collision avoidance	MQTT negotiation protocol	Deterministic, no extra messages, works without server
Proportional controller (no PID)	PID controller	Sufficient for straight aisles; fewer tuning parameters
Thread-safe EventQueue	Direct callback into FSM	State machine stays single-threaded; only shared structure with MQTT thread
Nearest-idle-robot dispatch	Optimal scheduling	O(n) simplicity; adequate for proof-of-concept; trivially replaceable
Sensor fusion (compass + encoders)	Encoders only	Compass corrects cumulative yaw drift
Vanilla JS SPA (no framework)	React / Angular	No build step; sufficient for monitoring UI scope

10. Conclusion

The Warehouse Robots architecture achieves clean separation across three independent tiers. The robot tier is fully platform-independent — its only Webots coupling is two concrete interface implementations. The server tier coordinates the fleet through lightweight, formally verified mechanisms. The design deliberately favours simplicity over premature optimisation; each algorithmic choice is behind an interface or a single method, making it straightforward to substitute more sophisticated solutions.